



PROYECTO FINAL

Sistema ETL paralelo para el procesamiento y consolidación de ventas de múltiples sucursales

DATOS

Universidad: Instituto Tecnológico de Las Américas (ITLA)

Materia: Programacion Paralela

Docente: Erick Leonardo Perez

Sección: Lunes de 8:00 pm - 10:00pm y Miercoles de 8 pm a - 10 pm

Datos De los Estudiantes

- Ranniel Muñoz 2024-2567
- Kalil Francisco Camilo 2023-1153
- Mario De Jesus Suero 2024-0263

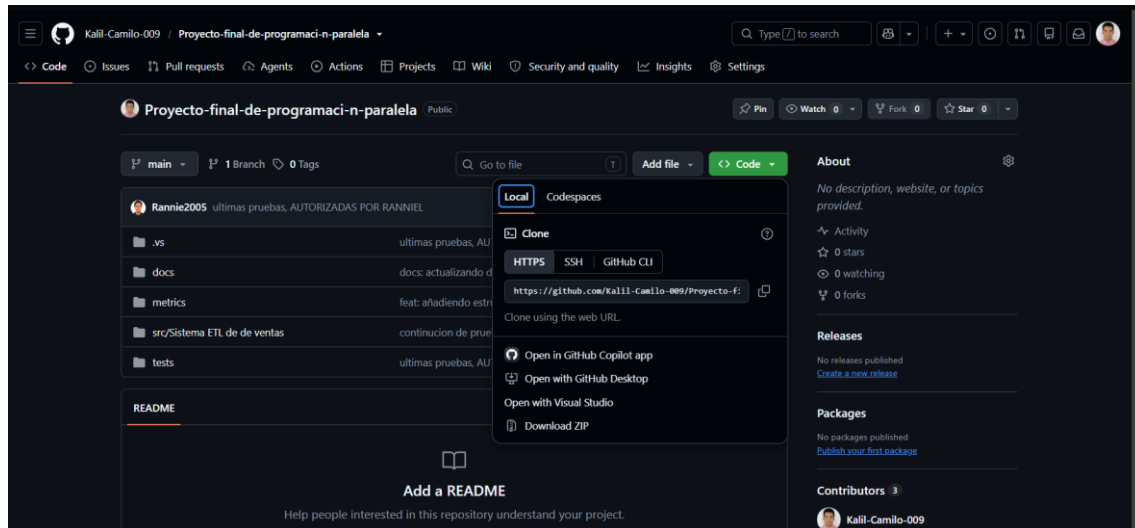
Fecha de entrega: 19/08/2026

Manual de Ejecución del Sistema

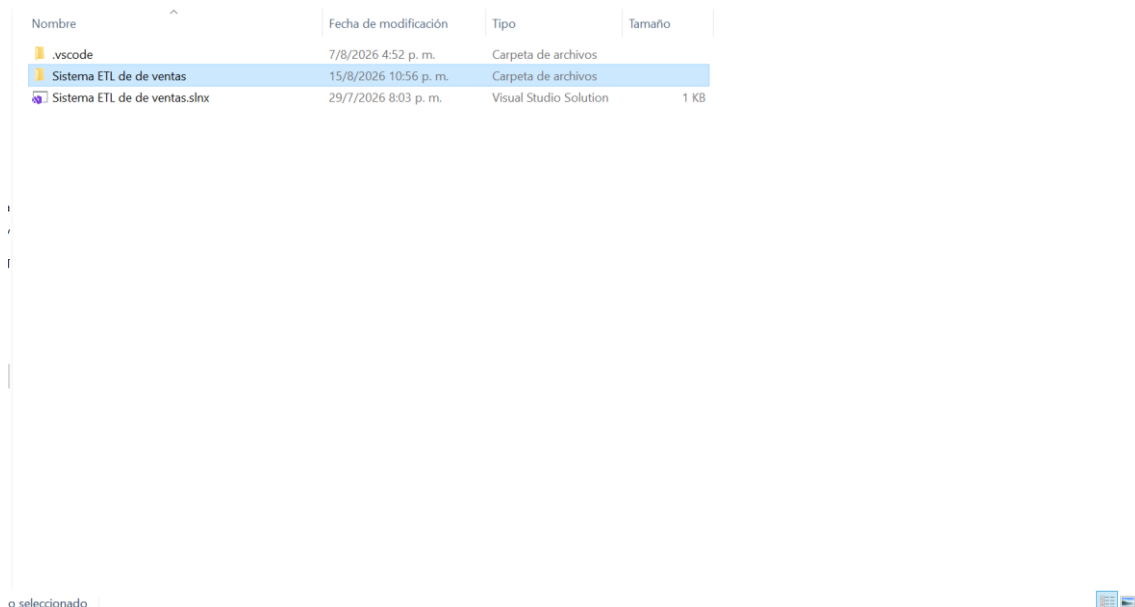
- Tiene que acceder al siguiente enlace del repositorio de GitHub.

<https://github.com/Kalil-Camilo-009/Proyecto-final-de-programaci-n-paralela>

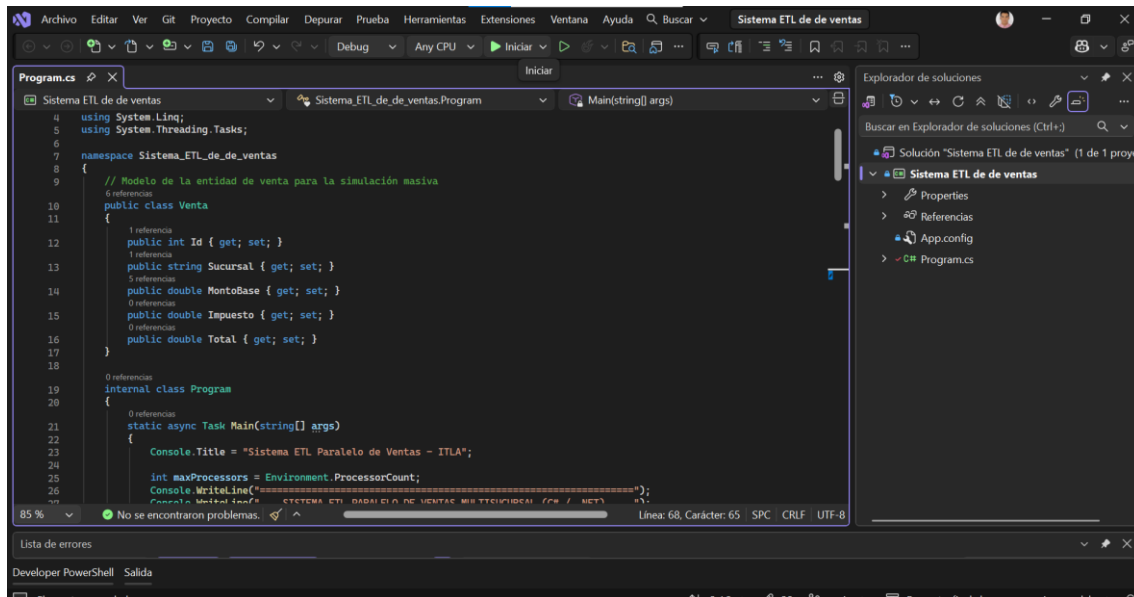
- Ya dentro del repositorio tendrás que ir al apartado en donde se encuentra un botón con el texto code, dale clic y después selecciona la opción de descargar zip.



- Ya dentro de la carpeta del proyecto, el usuario debería de poder localizar la carpeta /src y entrar directamente donde se encuentra el proyecto para poder abrirlo en Visual Studio.



- Ya con el proyecto abierto en Visual Studio, el usuario debería de dar clic en el botón de iniciar que está en el menú superior o simplemente oprimir el botón de F5 para poder ejecutar el proyecto.



- Después de que el programa está siendo ejecutado, el usuario deberá de ingresar una cantidad de procesadores disponibles que tenga su computador y después ingresar una cantidad masiva de datos a procesar.

```
=====
SISTEMA ETL PARALELO DE VENTAS MULTISUCURSAL (C# / .NET)
=====
Procesadores lógicos disponibles en el equipo: 4

Ingrese la cantidad de procesadores a utilizar: 1
Ingrese la cantidad de registros masivos a simular (ej. 10000000): 5000000

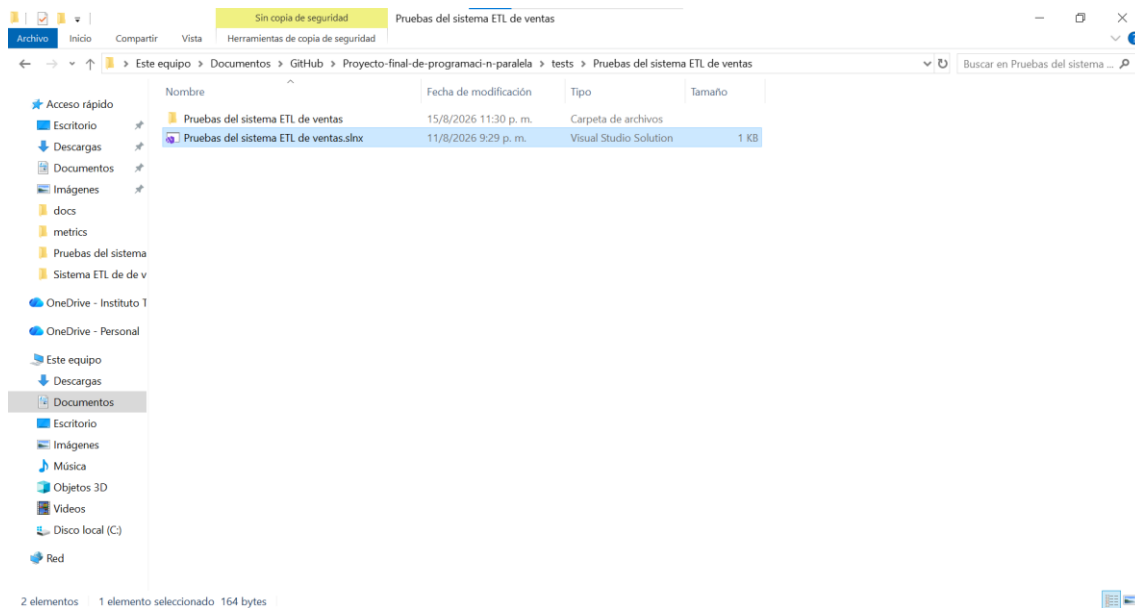
[1/3] Generando transacciones masivas en memoria...
[2/3] Ejecutando procesamiento ETL (Secuencial vs Paralelo)...

=====
RESULTADOS Y TELEMETRÍA
=====
Tiempo Secuencial: 72 ms
Tiempo Paralelo: 95 ms
Speedup (Aceleración): 0.76x
Eficiencia: 75.79%
¿Resultados coinciden?: True
=====

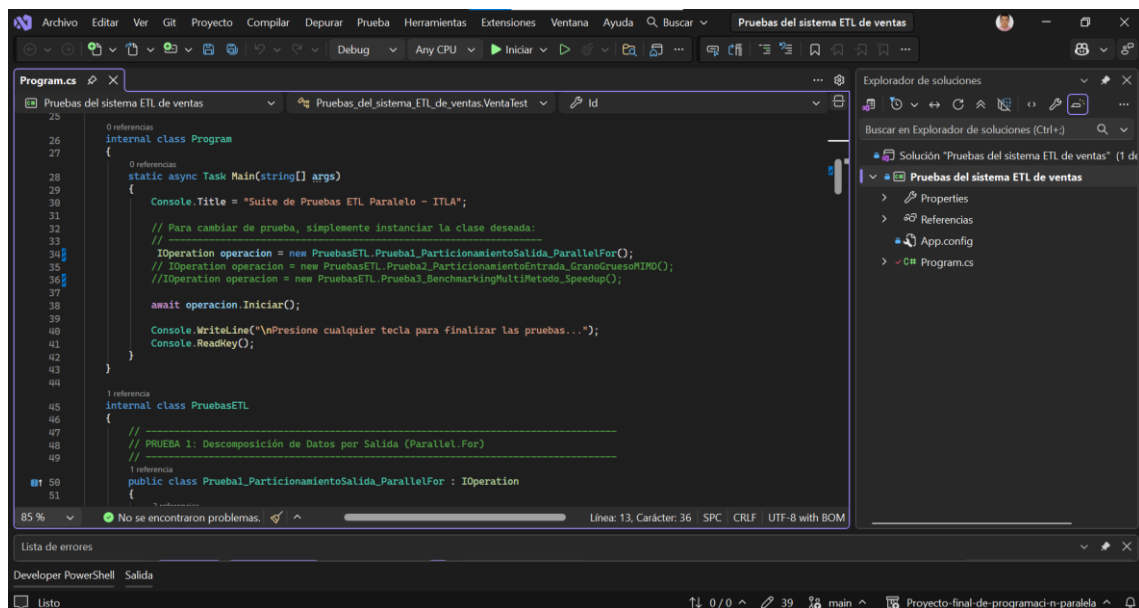
Proceso finalizado. Presione cualquier tecla para salir...

C:\Users\Kalil Camilo\Documents\GitHub\Proyecto-final-de-programaci-n-paralela\src\Sistema ETL de de ventas\Sistema ETL
de de ventas\bin\Debug\Sistema ETL de de ventas.exe (proceso 19308) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->
Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .
```

- Ya para en el caso de las pruebas, el usuario va a tener que ir a la carpeta llamada tests y abrir el proyecto en Visual Studio.



- El usuario debería situarse en el apartado de suite de pruebas del sistema de ventas ETL e ir comentando y descomentando las pruebas que quiere realizar.
- Básicamente, esto es un apartado que contiene una especie de suite de pruebas de rendimiento y procesamiento en paralelo para el sistema ETL de ventas, en donde lo desarrollamos que esté compuesto de tres pruebas principales.



- La primera prueba está encargada principalmente de transformar un conjunto de datos, en este caso serían 10 millones de ventas, utilizando el bucle de `parallel.for`.

The screenshot shows the Visual Studio IDE with the file `Pruebas_del_sistema_ETL_de_ventas.VentaTest` open. The code implements a test for data partitioning using `Parallel.For`. The console output shows the test results.

```
Program.cs
1 50 public class Prueba1_ParticionamientoSalida_ParalelFor : IOperation
2 51 {
3 52     public async Task Iniciar()
4 53     {
5 54         Console.WriteLine("=== PRUEBA 1: Particionamiento de Datos de Salida (Parallel.For) ===");
6 55         int N = 10_000_000;
7 56         VentaTest[] datos = GenerarDatos(N);
8 57         VentaTest[] resultado = new VentaTest[N];
9 58
10 59         Stopwatch sw = Stopwatch.StartNew();
11 60         Parallel.For(0, N, i =>
12 61         {
13 62             double itbis = datos[i].MontoBase * 0.18;
14 63             resultado[i] = new VentaTest
15 64             {
16 65                 Id = datos[i].Id,
17 66                 Sucursal = datos[i].Sucursal,
18 67                 MontoBase = datos[i].MontoBase,
19 68                 Impuesto = itbis,
20 69                 Total = datos[i].MontoBase + itbis
21 70             };
22 71         });
23 72         sw.Stop();
24 73
25 74         Console.WriteLine($"Procesados {N} registros de salida.");
26 75         Console.WriteLine($"Tiempo de ejecución paralelo: {sw.ElapsedMilliseconds} ms");
27 76     }
28 77
29 78 }
```

```
Consola de depuración de Microsoft Visual Studio
=== PRUEBA 1: Particionamiento de Datos de Salida (Parallel.For) ===
Procesados 10,000,000 registros de salida.
Tiempo de ejecución paralelo: 2860 ms

Presione cualquier tecla para finalizar las pruebas...

C:\Users\Kallil Camilo\Documents\GitHub\Proyecto-final-de-programaci-n-
-paralela\tests\Pruebas del sistema ETL de ventas\Pruebas del sistema
ETL de ventas\bin\Debug\Pruebas del sistema ETL de ventas.exe (proce
so 4776) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuració
n, habilite Herramientas -> Opciones -> Depuración -> Cerrar la consola
automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .
```

- La segunda prueba se encarga de procesar 20 millones de datos dividiéndolos todo en arreglos de bloques grandes, tantos bloques como núcleos que tenga el procesador mediante tareas usando `Task.run`. En palabras más sencillas, en lugar de trabajar en elemento por elemento, divide la lista de ventas en partes iguales según la cantidad de procesadores que tenga la computadora, aplicando principalmente el enfoque de grano grueso y la arquitectura MIMD.

The screenshot shows the Visual Studio IDE with the file `Pruebas_del_sistema_ETL_de_ventas.Program` open. The code implements a test for data partitioning using `Task.Run`. The console output shows the test results.

```
Program.cs
66 Console.WriteLine("=== PRUEBA 2: Particionamiento de Entrada (Grano Grueso - Arquitectura MIMD) ===");
67 int N = 20_000_000;
68 VentaTest[] datos = GenerarDatos(N);
69
70 int numProcesadores = Environment.ProcessorCount;
71 int chunkSize = N / numProcesadores;
72 var tareas = new Task<double>[numProcesadores];
73
74 Stopwatch sw = Stopwatch.StartNew();
75
76 for (int t = 0; t < numProcesadores; t++)
77 {
78     int taskIndex = t;
79     int start = taskIndex * chunkSize;
80     int end = (taskIndex == numProcesadores - 1) ? N : (taskIndex + 1) * chunkSize;
81
82     // Cada tarea ejecuta una instrucción/bucle autónomo sobre un subrango de datos (MIMD)
83     tareas[taskIndex] = Task.Run(() =>
84     {
85         double sumLocal = 0;
86         for (int i = start; i < end; i++)
87         {
88             sumLocal += datos[i].MontoBase * 1.18;
89         }
90         return sumLocal;
91     });
92 }
93
94 double[] resultadosParciales = await Task.WhenAll(tareas);
95 double granTotal = resultadosParciales.Sum();
96 sw.Stop();
97
98 Console.WriteLine($"Total procesado {N} en {numProcesadores} tareas MIMD: {granTotal:C2}");
99 Console.WriteLine($"Tiempo de ejecución: {sw.ElapsedMilliseconds} ms");
100 }
```

```
Consola de depuración de Microsoft Visual Studio
=== PRUEBA 2: Particionamiento de Entrada (Grano Grueso - Arquitectur
a MIMD) ===
Total procesado (N=20,000,000 en 4 tareas MIMD): $69,161,057,227.54
Tiempo de ejecución: 155 ms

Presione cualquier tecla para finalizar las pruebas...

C:\Users\Kallil Camilo\Documents\GitHub\Proyecto-final-de-programaci-n-
-paralela\tests\Pruebas del sistema ETL de ventas\Pruebas del sistema
ETL de ventas\bin\Debug\Pruebas del sistema ETL de ventas.exe (proce
so 18748) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuració
n, habilite Herramientas -> Opciones -> Depuración -> Cerrar la consola
automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .
```

- Por último, estaría la tercera prueba, que se puede decir que es como una especie de competencia y comparativa del rendimiento entre cuatro métodos distintos sobre 15 millones de datos que se están registrando, en donde se ejecutarían exactamente el mismo cálculo en cuatro enfoques distintos, con el enfoque secuencial, otro con el parallel for, uno utilizando tasks para el paralelismo por bloques, usando tareas asincrónicas y threads nativos, en donde el paralelismo está siendo creado y administrado directamente en hilos.

The screenshot shows a Visual Studio IDE with a C# program named 'Pruebas del sistema ETL de ventas'. The program is benchmarking four different execution methods: Sequential, Parallel For, Tasks (TPL), and Threads. The console output provides a detailed comparison of their performance.

```

// PRUEBA 3: Benchmarking Completo con Medición de Speedup, Eficiencia y Threads
//
1 referencia
public class Prueba3_BenchmarkingMultiMetodo_Speedup : IOperation
{
    2 referencias
    public async Task Iniciar()
    {
        Console.WriteLine("=== PRUEBA 3: Benchmarking Multi-método (Parallel, Tasks, Threads) ===");
        int maxProcessors = Environment.ProcessorCount;
        Console.WriteLine($"Procesadores del sistema: {maxProcessors}");

        int N = 15,000,000;
        Console.WriteLine($"Generando {N:N0} datos de prueba...");
        VentaTest[] datos = GenerarDatos(N);

        // 1. Ejecución Secuencial
        Stopwatch swSec = Stopwatch.StartNew();
        double totalSecuencial = 0;
        for (int i = 0; i < N; i++)
        {
            totalSecuencial += datos[i].MontoBase * 1.18;
        }
        swSec.Stop();
        long tSec = swSec.ElapsedMilliseconds;

        // 2. Ejecución con Parallel.For
        Stopwatch swParallel = Stopwatch.StartNew();
    }
}

```

Consola de depuración de Microsoft Visual Studio

```

=== PRUEBA 3: Benchmarking Multi-método (Parallel, Tasks, Threads) ===
Procesadores del sistema: 4
Generando 15,000,000 datos de prueba...

===== TABLA RECAPITULATIVA DE PRUEBAS =====
===== Tiempo Secuencial: 469 ms =====

Método: Parallel.For
Tiempo: 192 ms
Speedup: 2.44x
Eficiencia: 61.07%
Resultados coinciden: False

Método: Tasks (TPL)
Tiempo: 69 ms
Speedup: 6.80x
Eficiencia: 169.93%
Resultados coinciden: False

Método: Threads
Tiempo: 123 ms
Speedup: 3.81x
Eficiencia: 95.33%
Resultados coinciden: False

Presione cualquier tecla para finalizar las pruebas...

```

85 % No se encontraron problemas. Línea: 36, Carácter: 1

Developer PowerShell Salida

Listo

0/0 39 main Proyecto-final-de-programaci-n-paralela